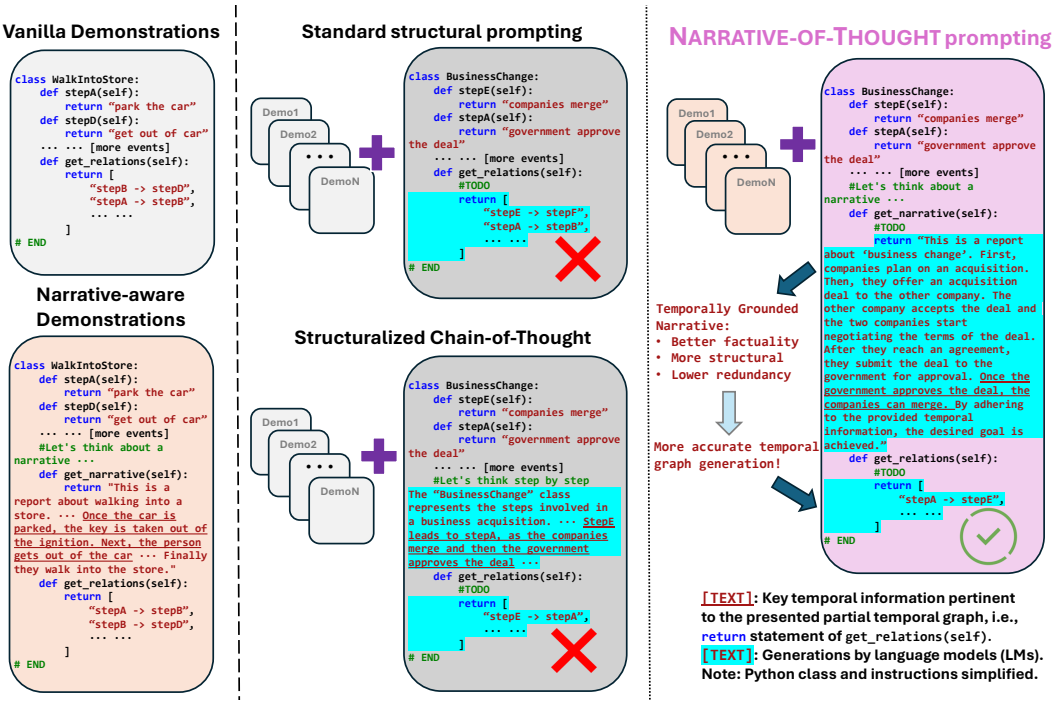

Figure 2: Overview of NARRATIVE-OF-THOUGHT (NOT), a prompting technique tailored for temporal reasoning. NOT improves the temporal graph by *recounting* a temporally grounded narrative. Also shown are comparisons with existing methods. Full example is in Figure A4 and NOT output is in Figure A7.

on **structural output generation**, augmented with producing a rationale in the form of a compelling and pertinent narrative.[4]

3 Method: NARRATIVE-OF-THOUGHT

Figure 2 provides an overview of the proposed NARRATIVE-OF-THOUGHT (NOT) method, and draws a comparison against common prompting techniques. Overall, given a scenario and a set of events, NOT first converts the input into a Python class, then guides LLMs to produce a temporally grounded narrative by arranging events in the correct temporal order, leveraging LLMs' intrinsic temporal knowledge. Based on the *recounted* temporal relations articulated in the narrative, LLMs are instructed to sort events into a temporal graph. This section will discuss major components in detail: (1) structural representation, (2) NOT prompting template, and (3) narrative-aware demonstrations.

Structural Representation. Following prior work (Madaan et al., 2022; Chen et al., 2022; Gao et al., 2023), we cast temporal reasoning as a code completion task. This design decision is motivated by the unordered nature of both event sets and temporal relation sets, making a structural representation the optimal choice. Wang et al. (2023a) also shows that combining structural event representations with LLMs trained with a mixture of text and code can unleash the full pretraining power. We extend this framing to handle cross-event structures. Specifically, a temporal graph is commonly presented in DOT format (Madaan and Yang, 2021; Sakaguchi et al., 2021), the appearance of which lends itself naturally to the usage of coding format. Furthermore, code execution follows a clear, step-by-step logical flow, mirroring the process of reasoning. Bringing these aspects together results in an alignment between temporal graphs and code structure, facilitating the temporal reasoning process. Our further study on this phenomenon also reveals a strong positive correlation between coding capabilities and temporal reasoning, as documented in Appendix E .

Concretely, each scenario is represented as a Python class. Each class encapsulates events as functions, where the function name is in the form of "step[A-Z]" such as "stepX", and the function body indicates the event description. The temporal graph is represented as a collection of pairwise temporal relations, enclosed within the return statement of "get_relation()" function, marked by "TODO" for LLMs to implement.

[4]The significance of narrative in shaping human decision-making is well-studied (Piper et al., 2021; Emelin et al., 2021; Zhang et al., 2024b); we hypothesize machines are similarly influenced.

NARRATIVE-OF-THOUGHT (NOT). At inference time, NOT first prompts LLMs to produce a temporally grounded narrative using *Narrative Prompt*. Drawing on the generated narrative, LLMs proceed and complete generation in response to *Temporal Graph Prompt*. The entire generation process is in an end-to-end manner, ensuring that LLMs explicitly leverage the temporal relations articulated in the narrative to assist the generation of the final temporal graph. We provide a complete example in Appendix C.

Narrative Prompt

```
# Let's think of a narrative to link aforementioned
events in the correct temporal order.
def get_narrative(self):
# TODO
```

Temporal Graph Prompt

```
def get_relations(self):
# TODO
# END
```

Overall, NOT narrows the gap between pre-training and inference by allowing the LLM to unfold the narrative knowledge seen during pre-training. Concretely, our approach leverages LLMs' inherent strengths in *generating* and *comprehending* text for narrative and temporal graph generation, respectively. In contrast, directly mapping abstract events to a temporal graph is less effective, as such examples are rarely encountered during pre-training. Practically, generated narratives create imagined experiences to navigate, and reify implicit timelines, assisting reasoning over a series of events even without explicit timestamps provided in the text, which are crucial for tasks requiring temporal reasoning. By reading the *recounted* narrative, it becomes easier for the LLMs to construct an implicit timeline to guide event sorting, significantly reducing the reasoning complexity compared to generating temporal graphs from scratch (i.e., using abstract events alone).

Our NOT draws a clear distinction from the CoT prompting and its variants in four aspects. First, for CoT, a final answer cannot be easily extracted unless a post-hoc script is designed (Kojima et al., 2022; Wang et al., 2023b; Zheng et al., 2024), which can be sometimes error-prone, while the output of NOT is easy to obtain by parsing the `get_relations()` function. Second, NOT produces final outputs in the structural space, while existing methods solely produce *simple, atomic outputs* as discussed in §2.2. Third, NOT produces final temporal graphs cost-effectively without external tools in an end-to-end fashion, unlike pipeline approaches which face error propagation and oversampling issues (Dror et al., 2023). Lastly, the generated rationales by CoTs are not necessarily grounded in real-world experience. In contrast, generated narratives by NOT are steered to be more *temporally grounded*, creating an imagined experience for LLMs to navigate, which is proved effective.

Narrative-aware Demonstrations. Existing studies (Brown et al., 2020; Wei et al., 2022a) have demonstrated that in-context demonstrations play a critical role in guiding LLMs to produce meaningful outputs. NOT is no exception, as Table 1 reveals that even GPT-3.5 struggles with temporal reasoning in a zero-shot setting. Thus, few-shot examples are provided by default. For NOT to succeed, high-quality and relevant rehearsed narratives, termed *reference narratives*, need to be created and embedded in these demonstrations.

Capitalizing on the recent success of using LLMs to generate demonstrations (Yu et al., 2023; Li et al., 2023), we prompt GPT-3.5/4 to produce reference narratives. Concretely, for each demonstration, abstracted as $\mathcal{G}(\mathcal{V}, \mathcal{E})$, we feed both $\mathcal{V}$ and $\mathcal{E}$ into GPT-3.5/4, using our designed reference narrative generation templates, dubbed *meta prompts*. In total, we create 4 types of meta prompts covering diverse genres like news and children's stories. Additionally, when feeding $\mathcal{G}(\mathcal{V}, \mathcal{E})$ into GPT-3.5/4, we use two *input formats* to define a Python class (*alphabetical* like "stepX" in Figure A8 vs. descriptive like "pushPedal" in Figure A9). We later evaluate the usefulness of each meta prompt in §5.2. Details of meta prompts are documented in Appendix D.

4 Experiment

In this work, we focus on **Temporal Graph Generation (TGG)**, an essential task of temporal reasoning. Here, we discuss datasets, experimental setup, baselines, and evaluation metrics. We provide additional implementation details in Appendix A.

4.1 Dataset

In line with the literature, we use **ProScript** (Sakaguchi et al., 2021) as the major benchmark, where a temporal script is represented as a directed acyclic graph, which were collected from a diverse range of

Method	Proscript				Schema-11				WikiHow Script				Avg.	
	F1↑	GED↓	$k(\mathcal{G})$	Cons.↑	F1↑	GED↓	$k(\mathcal{G})$	Cons.↑	F1↑	GED↓	$k(\mathcal{G})$	Cons.↑	F1↑	GED↓
Baselines														
Random	14.0	1.47	1.00	7.8	19.4	3.91	1.00	7.8	14.2	0.06	1.00	8.8	15.9	1.81
GPT-3.5 (0-shot)*	18.4	2.25	1.06	38.6	30.1	4.48	1.27	30.2	17.2	2.80	1.11	40.8	21.9	3.18
GPT-3.5	43.4	1.71	1.07	38.8	62.8	3.30	1.36	50.2	31.0	1.58	1.10	35.4	45.7	2.20
GPT-4	63.9	1.64	1.02	61.4	44.1	7.97	0.64	46.3	43.0	1.71	1.04	48.5	50.3	3.77
GEMMA-7B (Mesnard et al., 2024)														
Standard Prompting	19.7	**2.35**	1.02	**20.4**	27.8	5.03	**1.03**	18.3	17.5	**2.88**	**0.96**	**17.3**	21.7	**3.42**
Chain-of-Thought	20.0	**2.35**	1.01	20.0	26.4	5.03	**1.03**	14.9	13.6	5.91	0.73	11.5	20.0	4.43
NOT (no reference)	20.0	2.47	**1.00**	17.3	27.9	**4.78**	1.09	18.1	15.2	5.03	0.81	13.9	21.0	4.09
NOT (alphabetical meta)	**21.8**	2.48	**1.00**	18.3	**36.0**	4.84	1.06	19.7	**17.9**	2.95	**0.96**	16.9	**25.2**	**3.42**
NOT (descriptive meta)	21.3	2.60	0.99	17.8	34.8	5.00	1.06	**20.8**	**17.9**	**2.88**	0.95	16.8	24.7	3.49
MISTRAL-7B (Jiang et al., 2023)														
Standard Prompting	30.7	2.16	1.05	22.3	35.3	4.55	1.12	29.1	**22.5**	**2.09**	1.11	**18.9**	29.5	2.93
Chain-of-Thought	29.8	2.66	**1.02**	22.1	35.2	5.33	0.94	30.5	20.5	2.59	1.10	17.4	28.5	3.53
NOT (no reference)	32.5	3.04	0.95	19.4	42.3	5.27	**1.00**	27.6	21.8	3.33	0.98	15.4	32.2	3.88
NOT (alphabetical meta)	35.2	**2.11**	**1.02**	22.4	50.9	4.30	1.03	**36.1**	21.7	2.49	**1.04**	14.8	35.9	2.97
NOT (descriptive meta)	**35.4**	2.14	**1.02**	**23.0**	**52.7**	**3.90**	1.06	32.5	22.1	2.53	**1.04**	15.1	**36.7**	**2.86**
LLAMA3-8B (AI@Meta, 2024)														
Standard Prompting	25.1	2.39	1.18	19.9	28.3	4.42	1.24	19.9	20.6	1.17	1.07	21.2	24.7	2.66
Chain-of-Thought	30.1	2.06	**1.00**	23.3	37.3	5.79	0.85	23.5	22.6	**0.99**	**1.02**	**24.3**	30.0	2.95
NOT (no reference)	35.5	1.88	**1.00**	25.3	52.6	**3.18**	1.12	35.0	25.4	**0.99**	**1.02**	20.9	37.8	**2.02**
NOT (alphabetical meta)	**39.5**	1.87	1.01	**28.8**	59.0	3.72	1.12	39.1	26.3	1.01	1.03	22.5	41.6	2.20
NOT (descriptive meta)	38.7	**1.86**	1.01	28.4	**61.5**	3.57	**1.09**	**45.6**	**26.5**	1.04	1.03	22.3	**42.2**	2.16

Table 1: Main results of base LLMs and strong baselines on TGG evaluation benchmarks (average of 3 runs). For each base model, best results are **bold**, and NOT's variants better than both Standard Prompting and CoT are highlighted. NOT results that outperform 5-shot GPT-3.5 and GPT-4 are in blue. Results that meet both criteria are in purple. On average, NOT boosts F1 metric over its base model by 16% to 71%, and sometimes improves the GED metric. NOT-augmented LLAMA3-8B achieves best overall F1 (63.5 F1 by 3-shot variant; Figure 3) and GED results on Schema-11. Also, it only trails GPT-3.5 and GPT-4 by 8% and 14% on average, while yielding a lower average GED. Full results in Table A1. *By default, 5-shot examples are provided, unless otherwise noted.

sources including ROCStories (Mostafazadeh et al., 2016), Descript (Wanzare et al., 2016), and Virtual home (Puig et al., 2018). We also adopt two other datasets to enrich the evaluated genres and domains, and make necessary changes for the TGG task: 1) **Schema-11** evaluation set (Dror et al., 2023), which contains human-curated event schemas for 11 newsworthy topics, such as *armed robbery* and *business change*; and 2) **WikiHow Script** corpus (Lyu et al., 2021), a collection of multilingual how-to articles depicting necessary steps performed in sequence to achieve a high-level goal, covering a wide range of daily activities. Dataset statistics are included in Table 2, and we provide detailed dataset processing steps in Appendix B.

4.2 Setup

As our goal is to study the capability and generalizability of existing LLMs, and our NOT without any fine-tuning, we assume no access to large-scale training sets except for few-shot demonstrations. Therefore, all experiments are conducted in a 5-shot setting. We provide analysis on the impact of the shots numbers in §5.2. We consider three base models to spotlight the compatibility and versatility of NOT. We include very recent, strong LLMs, showing promising results on various reasoning tasks and code completion tasks, MISTRAL-7B (Jiang et al., 2023), GEMMA-7B (Mesnard et al., 2024), and LLAMA3-8B (AI@Meta, 2024). For all base models, we use their instruction-fine-tuned versions for experiments.

Shown in Figure 2, we represent the event set as a suite of Python methods, by serializing the unordered event set. For each scenario, we randomly shuffle the input Python methods three times, and apply models to each shuffle with greedy decoding at inference. For NOT, we use *Simple Report*-style narratives by GPT-4, which are generated by following instructions to produce concise reports based on provided event descriptions and relations (see style details in Table A2).

4.3 Baselines

To showcase the effectiveness of NOT, for each base model we compare with standard structural prompting and structuralized chain-of-thought prompting (Figure 2). We also remove reference